

2026-10-01

equivR: Reproducible Cross-Runtime Equivalence for R Analytics

Yonglin Zhu 

Independent open-source project
zhuygln@gmail.com

Xing Cai 

Independent open-source project
caixingt@gmail.com

Executive Summary

R is becoming an important open-source option for clinical-trial analysis and regulatory submissions, but long-term reviewability remains a practical challenge. Recreating an old R environment can be difficult as packages, runtimes, and dependencies change. Containers and environment preservation help, but eventually some analytical workflows must be refactored or migrated rather than frozen in place.

equivR addresses that next step: how do we modernize trusted analytical code without losing the behavior that made the original workflow trustworthy?

The project combines LLM-assisted code modernization with independent, deterministic validation:

Generation proposes; validation decides.

In the first Phase 1 demonstration, a trusted SAS workflow will be modernized to R. The LLM may generate or repair the R implementation, but **equivR** will decide acceptance by comparing its outputs with pregenerated trusted SAS reference artifacts under an explicit equivalence contract. A second demonstration will apply the same workflow to legacy R -> modern R, showing that the approach supports both cross-runtime migration and long-term maintenance within R.

Over four months, the project will deliver an open-source **equivR** core under MPL-2.0, including equivalence contracts, deterministic dataset and analytical-result validation, structured diagnostics, a model-independent repair loop, R-friendly local/CI workflows, and reproducibility evidence.

The goal is not to build another universal translator. It is to make analytical modernization verifiable: code may evolve, models may change, and runtimes may differ, while acceptance remains tied to trusted analytical results.

Signatories

Project team

Yonglin Zhu - co-PI / co-lead; Senior Staff Scientist at SAS Institute. ORCID [0000-0003-0245-827X](#); zhuygln@gmail.com. Led development of a completed SAS-to-Python modernization framework using AI-assisted transformation, hybrid SAS/Python execution, and numerical-equivalence validation. Leads architecture, equivalence-contract design, and validation workflow implementation. Project: <https://github.com/zhuygln/equivR>.

Xing Cai - co-PI / co-lead; founding member of the R Working Group at PPD. ORCID [0009-0004-5983-3338](#); caixingt@gmail.com. Applied analytics experience with SAS, R, Python, statistical modeling, and data-science workflows; co-leads R-community use-case design, benchmark development, testing, documentation, and applied evaluation.

equivR is independent open-source work: it is not sponsored, funded, or endorsed by either co-PI's employer and does not depend on employer-owned code, data, or infrastructure.

Contributors / consulted. None at submission time; community contributions will be invited through the public repository and benchmark process. The design is informed by public R Consortium Submissions Working Group materials ([R Consortium 2026c](#), [2026b](#), [2026a](#)); citation does not imply endorsement.

The Problem

Recent R Consortium discussion on making R submissions reviewable for FDA highlights a practical challenge: reviewers may be able to receive R code but still struggle to recreate the sponsor's environment, install dependencies, or reproduce the submitted results ([R Consortium 2026a](#)). As R becomes a more important open-source option for clinical-trial analysis, reviewability and reproducibility become as important as whether the code runs at all.

The R Submissions Working Group is already addressing this problem. Its 2026 plan includes environment-preservation approaches such as containers and WebAssembly, and it also plans to expand the use of AI and automation in future submission pipelines ([R Consortium 2026b](#)).

Preserving an old environment, however, does not solve every long-term problem. R versions and packages change, legacy code becomes difficult to maintain, and organizations eventually need to refactor or migrate analytical workflows. At that point the question becomes:

Can we change the implementation while preserving the analytical behavior that made the original workflow trustworthy?

Today, teams usually answer this with project-specific comparison scripts and manual review. There is no common infrastructure for defining what must remain equivalent and then checking those requirements across legacy R, modern R, SAS, Python, or AI-assisted rewrites.

Large language models make modernization easier, but they cannot be trusted to validate their own output. **equivR** therefore targets the missing layer: LLM-assisted modernization combined with

independent equivalence validation, so analytical workflows can evolve while remaining reproducible, reviewable, and trustworthy.

The proposal

Overview

equivR will provide a practical way to modernize analytical code without giving up the trusted behavior of the original workflow.

The first target is SAS -> R. A trusted SAS workflow will be used to generate reference artifacts. An LLM will produce or revise an R implementation, and **equivR** will independently compare the R results with the trusted SAS reference. If required checks fail, the diagnostics are returned for another repair attempt. The candidate is accepted only when all declared checks pass.

Generation proposes; validation decides.

This approach builds on our previous SAS -> Python modernization work, but the new project is centered on R and on a reusable validation workflow rather than on a single translation system.

Over four months, we will:

1. define a machine-readable equivalence contract;
2. implement deterministic comparison of datasets and selected analytical results;
3. demonstrate SAS -> R modernization using pregenerated SAS reference artifacts;
4. demonstrate legacy R -> modern R using the same validation framework;
5. connect validation diagnostics to a model-independent LLM repair loop; and
6. provide an R-friendly workflow, CI example, documentation, and an initial release.

For the R community, the benefit is a second path to long-term reviewability. Instead of preserving an old runtime indefinitely, a workflow can be modernized while the trusted implementation remains the reference for what must not change.

Detail

The project separates code generation from acceptance.

The LLM is responsible for proposing a refactor, translation, or repair. **equivR** is responsible for deciding whether that proposal is analytically equivalent to the trusted reference.

The basic workflow is shown in Figure 1.

The first demonstration will use a trusted SAS workflow and an R candidate. The second will use legacy R and modern R. Together, these cases show that the same validation layer can support both cross-runtime migration and long-term maintenance within R.

Minimum Viable Product

The Phase 1 MVP will allow a user to:

1. provide trusted reference artifacts and source code to be modernized;
2. generate or revise an R candidate with a selected LLM;
3. declare what must remain equivalent, including:
 - row keys and identity;

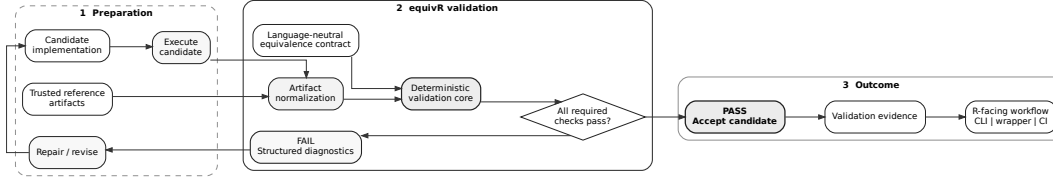


Figure 1: equivR modernization and validation workflow. An LLM (or developer) proposes a candidate implementation, which is executed and normalized alongside trusted reference artifacts. The deterministic core applies the equivalence contract and accepts the candidate only when all required checks pass; otherwise it returns structured diagnostics for repair and re-validation.

- required variables and schema;
 - missing-value behavior;
 - ignored metadata;
 - exact categorical values;
 - numerical tolerances; and
 - selected results such as estimates, confidence intervals, counts, and p-values;
4. execute the R candidate and normalize its outputs;
 5. receive deterministic PASS/FAIL status with structured diagnostics;
 6. send failed diagnostics back to the LLM for another repair attempt; and
 7. record machine-readable and human-readable validation evidence.

Two public examples will be delivered:

- SAS -> R, using pregenerated SAS reference artifacts so a SAS runtime is not required in CI; and
- legacy R -> modern R, showing that the same workflow applies to R maintenance and refactoring.

A contribution format will allow additional community cases to be added later.

Architecture

Phase 1 has four main components.

1. LLM modernization interface. A model-independent interface sends source code, task context, and validation diagnostics to an LLM and receives a proposed implementation or repair. equivR will not depend on one model provider or model family.

2. Equivalence contract. A machine-readable contract defines what behavior must be preserved. This makes acceptance criteria explicit instead of scattering them across one-off scripts and manual review.

3. Runtime and artifact layer. The candidate is executed and its outputs are converted into a common form for comparison. Trusted SAS artifacts may be generated ahead of time, so the proprietary runtime does not need to be present in CI.

4. Deterministic validator and evidence. The validator applies the contract to the trusted and candidate artifacts. Any failed required check blocks acceptance and produces structured diagnostics. Passing runs produce reproducibility metadata and validation evidence that can be used locally or in CI.

The R-facing interface will be kept thin: an R-friendly command or wrapper will invoke the workflow without requiring users to interact directly with the internal orchestration layer.

Assumptions

The project depends on a few explicit assumptions.

- A trusted reference implementation or trusted reference artifacts are available.
- The behavior that matters can be expressed through datasets and selected analytical results.
- Acceptable differences can be stated explicitly, including numerical tolerances where exact equality is not appropriate.
- General-purpose LLMs are capable of producing useful SAS -> R and legacy R -> modern R candidates when given source code and repair diagnostics.

If an LLM cannot complete a transformation, the validation layer still remains useful: it can evaluate human-written or externally generated candidates. If a behavior cannot be represented by the Phase 1 contract, that behavior will be documented as out of scope rather than treated as verified.

Phase 1 deliberately limits scope to small, testable examples and artifact-based validation. It does not attempt to prove universal program equivalence or cover every SAS/R feature.

External dependencies

The project will use open-source R and Python libraries, standard CI tooling, and existing general-purpose LLMs.

The automated modernization loop requires access to an LLM, but not to any particular vendor. The interface will allow different capable hosted or local models to be used.

SAS is needed only to create the trusted reference artifacts for the initial SAS -> R example. The shipped CI workflow will use pregenerated artifacts and will not require licensed SAS software.

Any proprietary model API costs used during development or evaluation will be self-funded or provided in kind rather than charged to the ISC grant.

Project plan

The project will run for four months. We will build the deterministic validation core first, demonstrate it on SAS -> R, then apply the same workflow to legacy R -> modern R, and finish with the LLM repair loop, documentation, and release.

Start-up phase

Month 1. Finalize project governance and MPL-2.0 licensing, define the first equivalence-contract format, and implement the initial deterministic validator for keys, schema, missing values, categorical values, numerical tolerances, and selected analytical results.

In parallel, prepare the first SAS -> R benchmark and pregenerate trusted SAS reference artifacts so public CI does not require a SAS runtime.

By the end of Month 1, **equivR** should accept a reference artifact, an R candidate result, and an equivalence contract, then return deterministic PASS/FAIL status with structured diagnostics.

Technical delivery

M1 - Equivalence contract and deterministic validation (Month 1-2 — \$1,250). Implement the reusable contract, dataset/result comparison, structured diagnostics, and tests. Any failed required check must block acceptance.

M2 - SAS -> R modernization demonstration (Month 1-2 — \$1,000). Use pregenerated SAS outputs as the trusted reference. An LLM produces or repairs an R implementation; **equivR** executes, normalizes, and validates the R outputs against the SAS reference. If the LLM cannot complete the transformation automatically, the candidate may be manually repaired. The validator remains the core deliverable.

M3 - Legacy R -> modern R and R-facing workflow (Month 2-3 — \$1,000). Apply the same workflow to a legacy R refactoring case. Add an R-friendly command or thin wrapper, a reference CI workflow, and a documented format for future benchmark contributions. This demonstrates that the approach generalizes beyond SAS migration to long-term maintenance of R code.

M4 - Model-independent repair loop and reproducibility evidence (Month 3-4 — \$750). Connect failed validation diagnostics to a bounded LLM repair loop:

```
generate/repair -> execute -> validate
                    PASS -> accept
                    FAIL -> diagnose -> repair again
```

The LLM interface will be model/provider independent. Validation remains deterministic and is the only authority for acceptance. Record validation results, contract version, checksums, run-time/session information, source revision, and model metadata when an LLM is used.

M5 - Documentation and release (Month 4 — \$1,000). Complete user and architecture documentation, SAS -> R and legacy R -> modern R examples, benchmark contribution guidance, tests, CI, and an initial tagged release.

Failure recovery. If an LLM cannot produce a correct candidate, we narrow or manually repair the example while preserving the validation experiment. Unsupported SAS/R behavior will be documented rather than treated as verified. Model-provider failure will not affect deterministic CI, and the repair loop will stop after a bounded number of attempts.

Other aspects

Development will be public under MPL-2.0, with community feedback and benchmark contributions through the repository.

Phase 1 remains intentionally narrow: two small benchmark cases, artifact-based validation, and no requirement for live SAS in CI. Future cases can extend the same framework to additional SAS -> R, Python -> R, package-migration, and other analytical modernization workflows.

No licensed SAS software, proprietary LLM subscription, cloud service, or AI credit will be charged to the grant.

Budget & funding plan

Milestone	Amount
M1 - Equivalence contract and deterministic validation	\$1,250
M2 - SAS -> R modernization demonstration	\$1,000
M3 - Legacy R -> modern R and R-facing workflow	\$1,000
M4 - Model-independent repair loop and reproducibility evidence	\$750
M5 - Documentation and release	\$1,000
Total	\$5,000

No grant funds are requested for travel, workshops, hardware, publication fees, cloud services, AI credits, indirect costs, or licensed SAS software.

Success

Definition of done

Phase 1 is complete when `equivR` can demonstrate the full modernization workflow in two cases:

1. SAS -> R: an R candidate is generated or repaired with an LLM and accepted only after deterministic validation against trusted SAS reference artifacts; and
2. legacy R -> modern R: the same validation workflow is used to refactor an existing R analysis.

For both cases, users must be able to define an equivalence contract, run validation locally or in CI, receive structured diagnostics for failures, and produce reproducible validation evidence.

A live SAS runtime will not be required in CI. The automated repair loop will be model/provider independent, and the LLM will never determine acceptance.

Measuring success

Success will be measured by whether:

- both benchmark cases run from a clean checkout;
- all declared required checks pass before a candidate is accepted;
- deliberately introduced mismatches fail with actionable diagnostics;
- the SAS -> R example works from pregenerated trusted artifacts;
- the same validation framework supports the legacy R -> modern R case;
- an R-friendly invocation path and CI example are documented;
- validation results include reproducibility metadata and machine-readable evidence; and
- an initial public release is published.

The goal is not to show that an LLM can translate every analytical program. The goal is to show that modernization can be made verifiable: generation may vary, but acceptance remains deterministic.

Future work

After Phase 1, the same framework can be extended to additional SAS -> R and Python -> R cases, richer analytical result types, live runtime adapters, larger repository-scale modernization, and more automated multi-step repair.

The benchmark format can also grow through community contributions and future R Consortium submission-oriented use cases.

R Consortium. 2026a. “Making R Submissions Reviewable for FDA.” 2026. <https://r-consortium.org/posts/making-r-submissions-reviewable-for-fda/>.

———. 2026b. “R Submissions Working Group: 2026 Plans.” 2026. <https://r-consortium.org/posts/submissions-wg-2026/>.

———. 2026c. “Submissions Pilot 5 Dataset-JSON Repository.” 2026. <https://github.com/RConsortium/submissions-pilot5-datasetjson>.